FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DISEÑO E IMPLEMENTACIÓN DEL SISTEMA DE GESTIÓN DE SOFTWARE PARA LA BODEGA CRUZ DE CHALPÓN

Autores:

CASTREJON RODRIGUEZ, Harold Cesar (100 %)

CENTURIÓN VIGO, José Wilfredo (100 %)

FRIAS TORRES, Luis Miguel (100 %)

MENDOZA TERRONES, Andres Jhonel (100 %)

OCAS MUÑOZ, Clemente Daniel (100 %)

Curso:

Calidad y Pruebas de Software

Docente:

URIARTE CORTEGANA, Kenny Jhoel

Cajamarca – Perú

2026 – 1

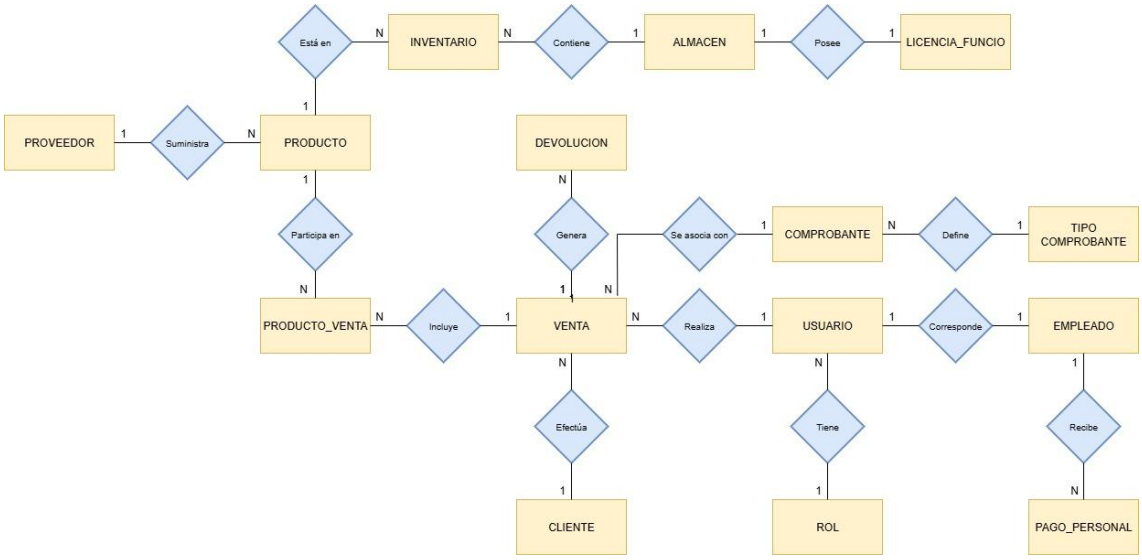
1. **Características técnicas del producto o Software a desarrollar**

El sistema está diseñado para optimizar los procesos administrativos y operativos de una bodega familiar. Sus principales características técnicas son:

- **Arquitectura:** Monolítica organizada en capas (Presentación, Lógica de Negocio y Acceso a Datos) bajo el patrón MVC (Modelo-Vista-Controlador).
- **Lenguaje de Programación:** C# utilizando la plataforma .NET.
- **Entorno de Desarrollo (IDE):** Visual Studio 2022.
- **Base de Datos:** SQL Server LocalDB (o SQL Server Express) para la persistencia de datos.
- **Atributos de Calidad (ISO/IEC 25010):** Se priorizaron la usabilidad (interfaz intuitiva para personal no técnico), seguridad (cifrado y control de accesos), mantenibilidad (módulos aislados) y disponibilidad (99.5% en horario comercial).
- **Despliegue:** Local (on-premise) para asegurar la operatividad ante conexiones inestables en zonas rurales.

2. **Diagrama de base de datos**

- Modelo conceptual de datos



- Modelo lógico

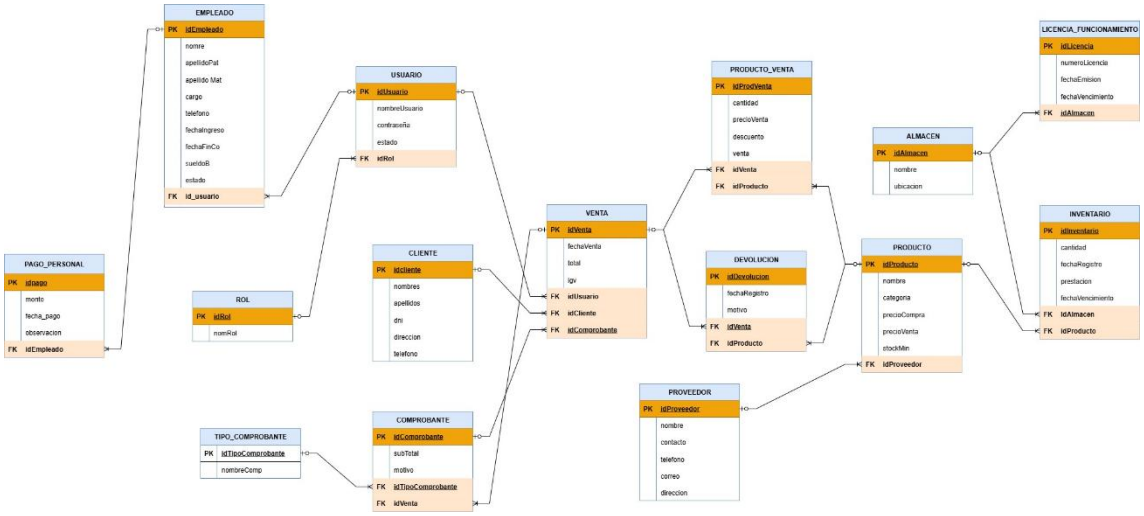
- Almacen.cs
  - Almacen
    - IdAlmacen : int
    - Nombre : string
    - Ubicacion : string
    - Productos : ICollection<Producto>
- Cliente.cs
  - Cliente
    - IdCliente : int
    - Nombres : string
    - Apellidos : string
    - DNI : int
    - Direccion : string
    - Telefono : int
    - Ventas : ICollection<Venta>
- Comprobante.cs
  - Comprobante
    - IdComprobante : int
    - SubTotal : decimal
    - Motivo : string
    - VentaId : int
    - Venta : Venta
    - ProductoId : int
    - Producto : Producto
    - IdTipoComprobante : int
    - TipoComprobante : TipoComprobante
- Devolucion.cs
  - Devolucion
    - IdDevolucion : int
    - FechaRegistro : DateTime
    - Motivo : string
    - VentaId : int
    - Venta : Venta
    - ProductoId : int
    - Producto : Producto

- Devolucion.cs
  - Devolucion
    - IdDevolucion : int
    - FechaRegistro : DateTime
    - Motivo : string
    - VentaId : int
    - Venta : Venta
    - ProductoId : int
    - Producto : Producto

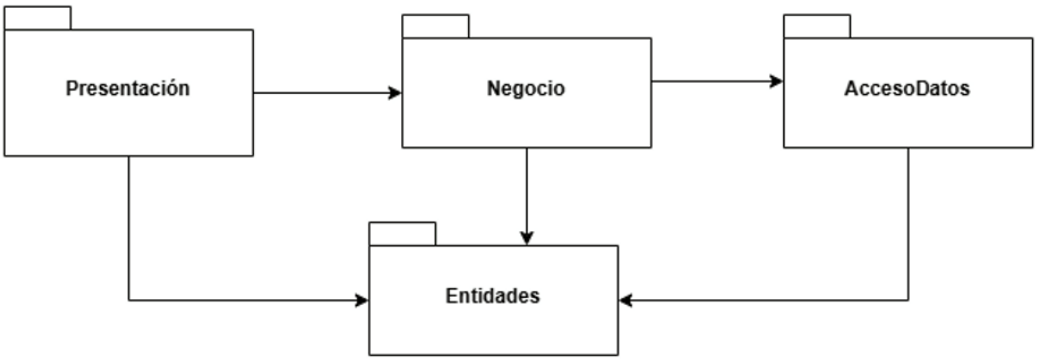
- Empleado.cs
  - Empleado
    - IdEmpleado : int
    - Nombre : string
    - ApellidoPat : string
    - ApellidoMat : string
    - Cargo : string
    - Telefono : int
    - FechaIngreso : DateOnly
    - FechaFinCo : DateOnly
    - SueldoB : decimal
    - Estado : bool
    - Usuarios : ICollection<Usuario>
    - Pagos : ICollection<Pago_Personal>
    - licencia_Funcionamientos : ICollection<Licencia_Funcionamiento>
- ErrorViewModel.cs
- Inventario.cs
  - Inventario
    - IdInventario : int
    - Cantidad : int
    - FechaRegistro : DateTime
    - Presentacion : string
    - FechaVencimiento : DateOnly
    - ProductoId : int
    - Producto : Producto
- Licencia_Funcionamiento.cs
  - Licencia_Funcionamiento
    - IdLicencia : int
    - NumeroLicencia : string
    - FechaEmision : DateTime
    - FechaVencimiento : DateTime
    - EmpleadoId : int
    - Empleado : Empleado

```
Proveedor.cs
    Proveedor
        IdProveedor : int
        Nombre : string
        Contacto : string
        Telefono : int
        Correo : string
        Direccion : string
        Productos : ICollection<Producto>
Rol.cs
    Rol
        IdRol : int
        NombreRol : string
        Usuarios : ICollection<Usuario>
TipoComprobante.cs
    TipoComprobante
        IdTipoComprobante : int
        NombreComp : string
        Comprobantes : ICollection<Comprobante>
Usuario.cs
    Usuario
        IdUsuario : int
        NombreUsuario : string
        Contraseña : string
        Estado : bool
        RolId : int
        Rol : Rol
        EmpleadoId : int
        Empleado : Empleado
        Ventas : ICollection<Venta>
Venta.cs
    Venta
        IdVenta : int
        FechaVenta : DateTime
        Total : decimal
        igv : decimal
        ClienteId : int
        Cliente : Cliente
        UsuarioId : int
        Usuario : Usuario
        ProductoVentas : ICollection<ProductoVenta>
        Comprobantes : ICollection<Comprobante>
        Devoluciones : ICollection<Devolucion>
```

❖ Modelo Físico



3. Diagrama de paquetes (Modelo Arquitectónico)



4. Resumen de Modelo OWASP y top 10 – de los últimos años

El **Modelo OWASP** (Open Web Application Security Project) es un marco de trabajo de una organización sin fines de lucro que proporciona recursos, herramientas y estándares para mejorar la seguridad del software a nivel global. Su proyecto más influyente es el **OWASP Top 10**, una lista que identifica los riesgos de seguridad más críticos para las aplicaciones web, basada en datos reales de miles de vulnerabilidades reportadas.

4.1. Control de Acceso Quebrado: Ocurre cuando las restricciones sobre lo que los usuarios autenticados pueden hacer no se aplican correctamente.

Ejemplo: Un usuario puede acceder a la cuenta de otro simplemente cambiando el ID en la URL, o un usuario común accede a funciones de administrador.

4.2. Fallos Criptográficos: Antes llamado "Exposición de Datos Sensibles". Se enfoca en la protección de datos en reposo y en tránsito.

Ejemplo: Guardar contraseñas en texto plano, usar algoritmos de cifrado obsoletos (como MD5 o SHA1) o no usar certificados TLS (HTTPS) actualizados.

4.3. Inyección: Ocurre cuando se envían datos no confiables a un intérprete como parte de un comando o consulta.

Ejemplo: Inyección SQL (acceso a bases de datos), inyección de comandos de SO o XSS (Cross-Site Scripting). Los datos del usuario "engañan" a la aplicación para ejecutar órdenes no deseadas.

4.4. **Diseño Inseguro:** Es una categoría nueva que se centra en los riesgos relacionados con fallos de diseño y arquitectura.

Ejemplo: Una aplicación que no tiene una función de "recuperación de contraseña" segura desde su concepción. No se soluciona programando mejor, sino diseñando mejor el flujo del negocio.

4.5. **Configuraciones de Seguridad Incorrecta:** Sucede cuando los ajustes de seguridad no se definen, se implementan mal o se dejan los valores por defecto.

Ejemplo: Dejar activadas funciones innecesarias, cuentas con contraseñas de fábrica o mostrar mensajes de error detallados que revelan información del servidor a los atacantes.

4.6. **Componentes vulnerables y Obsoletos:** Se refiere a aplicaciones que utilizan bibliotecas, frameworks o dependencias que tienen vulnerabilidades conocidas o ya no reciben soporte.

Ejemplo: Usar una versión antigua de WordPress o una librería de JavaScript que ya tiene un exploit publicado en internet.

4.7. **Fallos de Identificación y Autenticación:** Ocurre cuando las funciones relacionadas con el registro, inicio de sesión y gestión de sesiones permiten que un atacante suplante identidades.

Ejemplo: Permitir ataques de fuerza bruta (sin límite de intentos), permitir contraseñas muy débiles o no invalidar la sesión tras cerrar el navegador.

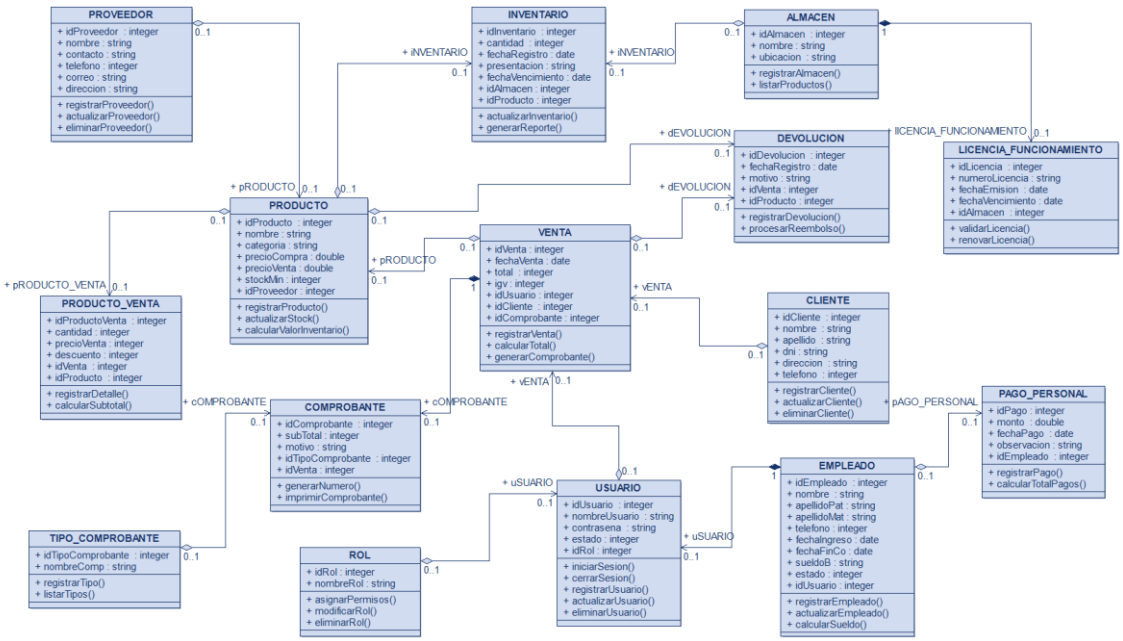
4.8. **Fallos de Integridad de Software y Datos:** Se centra en el código y los datos que se asumen como seguros sin verificar su integridad.

Ejemplo: Actualizaciones automáticas de software que no están firmadas digitalmente, permitiendo que un atacante inyecte código malicioso en el proceso de actualización.

4.9. **Fallos de Registros y Monitoreo:** Es la incapacidad de detectar y responder a brechas de seguridad en tiempo real.

Ejemplo: si un atacante intenta entrar 10,000 veces a una cuenta y el sistema no genera una alerta o no guarda un registro (log), el ataque pasa desapercibido hasta que es demasiado tarde.

4.10. Falsificación de Solicitudes del Lado del Servidor (SSRF): Ocurre cuando una aplicación web recupera un recurso remoto sin validar la URL proporcionada por el usuario.

Ejemplo: Un atacante obliga a la aplicación a hacer una petición HTTP a una dirección interna de la empresa (como un panel de control privado) que no debería ser accesible desde el exterior.

5. Diagrama de clases